



WEB TECHNOLOGIES

HTML 5 – Audio, Video and Progress elements

Prof. Pavan A C

Department of Computer Science and Engineering

Acknowledgement:

Teaching Assistants(Harini B and Chandana MS)

HTML5 - Audio

Audio elements

- A standard approach for browser to play audio without the use of plug-in.
- Mp3, Wav and ogg formats supported by HTML5

Syntax

```
<audio controls="c1">  
  <source src="song.ogg" type="audio/ogg" />  
  <source src="song.mp3" type="audio/mp3" />  
  Your browser does not support the audio element.  
</audio>
```

HTML5 - Audio

Audio element properties

Attribute	Value	Description
autoplay	autoplay	Specifies that the audio will start playing as soon as it is ready.
controls	controls	Specifies that controls will be displayed, such as a play button.
loop	loop	Specifies that the audio will start playing again (looping) when it reaches the end
preload	preload	Specifies that the audio will be loaded at page load, and ready to run. Ignored if autoplay is present.
src	<i>url</i>	Specifies the URL of the audio to play

- A standard approach for browser to play video without the use of plug-in
- Any content between the opening and closing tags is fallback content. This content is displayed only by browsers that don't support the <video> tag

Syntax

```
<video width="320" height="240" controls="controls">  
  <source src="movie.ogg" type="video/ogg" />  
  <source src="movie.mp4" type="video/mp4" />  
  <source src="movie.webm" type="video/webm" />  
  Your browser does not support the video tag.  
</video>
```

HTML5 - Video

Video element properties

Attribute	Value	Description
audio	muted	Defining the default state of the the audio. Currently, only "muted" is allowed
autoplay	autoplay	If present, then the video will start playing as soon as it is ready
controls	controls	If present, controls will be displayed, such as a play button
height	<i>pixels</i>	Sets the height of the video player
loop	loop	If present, the video will start over again, every time it is finished
poster	<i>url</i>	Specifies the URL of an image representing the video
preload	preload	If present, the video will be loaded at page load, and ready to run. Ignored if "autoplay" is present
src	<i>url</i>	The URL of the video to play
width	<i>pixels</i>	Sets the width of the video player

- The `<progress>` tag represents the completion progress of a task.
- Always add the `<label>` tag for describing the task!
- Use JavaScript to manipulate the value of the progress bar

Syntax:

```
<label for="file">Downloading progress:</label>
```

```
<progress id="file" value="32" max="100"> 32% </progress>
```

Attribute	Value	Description
<u>max</u>	<i>number</i>	Specifies how much work the task requires in total. Default value is 1
<u>value</u>	<i>number</i>	Specifies how much of the task has been completed

1. Which HTML5 tag is used to embed audio files in a web page?
 - A) <video>
 - B) <sound>
 - C) <audio>
 - D) <music>
2. In the <audio> tag, which attribute is used to display the browser's default audio controls?
 - A) autoplay
 - B) controls
 - C) loop
 - D) muted
3. What happens to the content placed between <video> and </video> tags if the browser does not support the <video> tag?
 - A) It is ignored.
 - B) It acts as fallback content and is displayed.
 - C) It throws an error.
 - D) It is automatically converted to audio.

4. Which attributes are used in the <progress> tag to indicate task completion?

- A) loop and autoplay
- B) src and controls
- C) max and value
- D) muted and preload

5. What is an “indeterminate” progress bar in HTML5?

- A) A progress bar without color
- B) A progress bar showing cyclic movement without exact progress value
- C) A vertical progress bar
- D) A disabled progress bar

HTML5 -

MCQ - Answers

Answer 1: C) <audio>

Answer 2: B) controls

Answer 3: B) It acts as fallback content and is displayed.

Answer 4: C) max and value

Answer 5: B) A progress bar showing cyclic movement without exact progress value



THANK YOU

Prof. Pavan A C

Department of Computer Science and Engineering



PES

UNIVERSITY

CELEBRATING 50 YEARS

WEB TECHNOLOGIES

HTML 5 – Geo Location

Prof. Pavan A C

Department of
Computer Science and Engineering

Acknowledgement:

Teaching Assistants(Harini B and Chandana MS)

- Enables your web application to obtain the geographical position of your website visitors
- The user has to accept to share their location,
- The Geolocation API is work with the `navigator.geolocation` object.
- Accessed via JavaScript, through the `navigator.geolocation` object.
- `navigator.geolocation` object allows you to access geo location through two primary functions:
 - `getCurrentPosition()`
 - Returns the location of the visitor as a one-time snapshot
 - `watchPosition()`
 - returns the location of the visitor every time the location changes

Syntax : `getCurrentPosition(success, error, options)`
`watchPosition(success, error, options)`

- Both functions take the following parameters:
 - Success callback function
 - Error callback function (optional)
 - Geo location options object (optional)
 - If the `getCurrentPosition()` method is successful, it returns a coordinates object to the function specified in the parameter (success)

HTML5 – Geo Location

Introduction

The Geolocation API uses three methods of Geolocation interface which are given following:

Methods	Description
<code>getCurrentPosition()</code>	It identifies the device or the user's current location and returns a position object with data.
<code>watchPosition()</code>	Return a value whenever the device location changes.
<code>clearWatch()</code>	It cancels the previous <code>watchPosition()</code> call

```
navigator.geolocation.getCurrentPosition(showPosition);  
function showPosition(position) {  
    x.innerHTML = "Latitude: " + position.coords.latitude +  
        "<br>Longitude: " + position.coords.longitude;  
}
```

```
navigator.geolocation.getCurrentPosition( function(position) {  
    console.log("your position is: " + position.coords.latitude + ", " +  
        position.coords.longitude);  
} );
```

* `getCurrentPosition` can be replaced with `watchPosition` if you need to receive updates to location

1. Which JavaScript object provides access to the HTML5 Geolocation API?

- A) window.location
- B) navigator.location
- C) navigator.geolocation
- D) document.geolocation

2. Which Geolocation API method returns the user's location as a one-time snapshot?

- A) getPosition()
- B) getCurrentPosition()
- C) watchPosition()
- D) trackPosition()

3. In the Geolocation API, the first parameter of getCurrentPosition() is:

- A) Error callback function
- B) Success callback function
- C) Options object
- D) Coordinates object

4. What does the position object returned by Geolocation API contain?

- A) Only latitude
- B) Only longitude
- C) Both latitude and longitude in coords property
- D) URL of user's location

5. If the user denies location permission, which callback is executed in the `getCurrentPosition()` method?

`navigator.geolocation.getCurrentPosition(successCallback, errorCallback);`

- A) successCallback
- B) errorCallback
- C) Both callbacks execute simultaneously
- D) None of them

Answer 1: C) navigator.geolocation

Answer 2: B) getCurrentPosition()

Answer 3: B) Success callback function

Answer 4: C) Both latitude and longitude in coords property

Answer 5: B) errorCallback



THANK YOU

Prof. Pavan A C

Department of Computer Science and Engineering



Acknowledgement:

Teaching Assistants(Harini B and Chandana MS)

WEB TECHNOLOGIES

Callback and Promises

Prof. Pavan A C

Department of
Computer Science and Engineering

Callbacks

Introduction



- A callback is a function passed as an argument to another function
 - This technique allows a function to call another function
 - They will be called asynchronously based on timer or other events
 - These function references are called **Callback functions**
 - As seen in setInterval, setTimeout and addEventListener, a function accepts a function reference as an argument.
-
- Example:
 - `div.addEventListener("keypress", function(){ ... });`

Callbacks

Introduction



```
function greet(name, callback) {  
    console.log("Hello, " + name);  
    callback();  
}  
function sayBye() {  
    console.log("Goodbye!");  
}  
greet("Ajay", sayBye);
```

In this example, the `sayBye` function is passed as an argument to the `greet` function and is then invoked from within `greet`. This demonstrates how functions in JavaScript can be passed as arguments to other functions—also known as **callbacks**. The `greet` function can accept **any function** as a callback, and it will execute that function after greeting the user.

Synchronous Calls

- Code executes one statement at a time, blocking further execution until the current statement finishes.

```
console.log("One");  
console.log("Two");  
console.log("Three");  
// Output: One  
//      Two  
//      Three
```

Asynchronous Calls

- Some tasks (like timers or network requests) don't complete right away. Asynchronous calls let your code schedule work to happen later, without blocking.

Example:

```
console.log("One");  
setTimeout(function() {  
  console.log("Two");  
}, 1000);  
console.log("Three");
```

// Output:

One

Three

// (waits 1 second...)

Two

Callback Hell

Callback hell (also called the "pyramid of doom") occurs when callbacks are nested within other callbacks many levels deep. This makes code hard to read, understand, and maintain.

Why does it happen?

- Asynchronous tasks are often dependent on the results of previous tasks.
- If you write each dependent task inside the previous callback, you end up with multiple levels of nesting.
- This nesting makes code:
 - a. Difficult to follow.
 - b. Harder to debug.

Callback Hell

```
doTask1(function(result1) {  
  doTask2(result1, function(result2) {  
    doTask3(result2, function(result3) {  
      doTask4(result3, function(result4) {  
        // ...callback hell!  
      });  
    });  
  });  
});
```

```
getUser(userId, (user) => {  
  getOrders(user, (orders) => {  
    processOrders(orders, (processed) => {  
      sendEmail(processed, (confirmation) => {  
        console.log("Order Processed:", confirmation);  
      });  
    });  
  });  
});
```

Promises

Introduction



"I Promise a Result!"

"Producing code" is code that can take some time

"Consuming code" is code that must wait for the result

A Promise is a JavaScript object that links producing code and consuming code

A JavaScript Promise object contains both the producing code and calls to the consuming code:

Promises

A Promise is a special JavaScript object used to handle asynchronous operations. It represents a value that may be available now, later, or never (in case of an error). Promises allow more readable async code compared to deeply nested callbacks (callback hell).

A Promise is a placeholder for the result of an asynchronous task.

- A Promise is in one of three states:
 - Pending: The operation is ongoing.
 - Fulfilled: The operation completed successfully.
 - Rejected: The operation failed.

Promises

Introduction

- Promise is a JavaScript object with a value that may not be available at the moment when the code line executes.
- These values are resolved at some point in the future.

While a Promise object is "pending" (working), the result is undefined.

When a Promise object is "fulfilled", the result is a value.

When a Promise object is "rejected", the result is an error object.

You cannot access the Promise properties state and result.

You must use a Promise method to handle promises.

- A promise is used to handle the asynchronous result of an operation.
- With Promises, we can defer execution of a code block until an async request is completed.
- The Promise object is created using the new keyword and contains the promise; this is an executor function which has a **resolve** and a **reject** callback
- Essentially, a promise is a returned object to which you attach callbacks, instead of passing callbacks into a function.

```
const promise = new Promise(function(resolve, reject) {  
  // promise description  
});
```

Promises

Introduction: Syntax for Callback and promises.



```
const myPromise = new Promise(function(resolve, reject) {  
  // Simulate async operation  
  setTimeout(function() {  
    let success = true; // Try changing to false to see rejection handling  
    if (success) {  
      resolve("Operation succeeded!");  
    } else {  
      reject("Operation failed.");  
    }  
  }, 1000);  
});
```

```
// Using the Promise  
myPromise  
  .then(function(result) {  
    console.log(result); // Runs if resolved: "Operation succeeded!"  
  })  
  .catch(function(error) {  
    console.log(error); // Runs if rejected: "Operation failed."  
  });
```

Promises

Introduction

```
// Promise constructor
let promise = new Promise(function(resolve, reject) {
    const x = "apple";
    const y = "apple"

    if (x === y) {
        resolve();
    } else {
        reject();
    }
});

// Consuming the Promise
promise
    .then(function () {
        console.log('Successful');
    })
    .catch(function () {
        console.log('Some error has occurred');
    });
```

- Promise constructor takes only one argument which is a callback function (and that callback function is also referred as an anonymous function too).
- Callback function takes two arguments, resolve and reject. Perform operations inside the callback function and if everything went well then call resolve.

Promises

Example



```
var weather;
const date = new Promise(
  function(resolve, reject) {
    weather = true; //usually a API call
    if (weather) {
      const dateDetails = {
        name: 'Cubana Restaurant',
        location: '55th Street',
        table: 5
      };
      resolve(dateDetails)
    } else {
      reject(new Error('Bad weather'))
    }
  }
);

date
  .then(function(done) {
    // This runs when resolve() is called
    console.log('We are going on a date!')
    console.log(done)
  })
  .catch(function(error) {
    // This runs when reject() is called
    console.log(error.message)
  })
```

Callback and Promises

Comparison

- Callbacks and Promises are not the same
- Callbacks are function passed to another function as a reference
- Chaining of Callbacks can be clumsy and lead to **Callback Hell**
- Promises use Callbacks and more elegant than Callbacks
- Chaining of Promises is supported

Callback and Promises

MCQ



1)Complete the code so that **sayBye** is called only after **greet** prints the greeting:

```
function greet(name, callback) {  
    console.log("Hello, " + name);  
    // _____  
}  
  
function sayBye() {  
    console.log("Goodbye!");  
}  
  
greet("Ajay", sayBye);
```

A) sayBye();

B) sayBye;

C) callback();

D) callback;

2) What will be the output of the following code?

```
console.log("First");
```

```
setTimeout(() => { console.log("Second"); }, 50);
```

```
console.log("Third");
```

- A) First Second Third
- B) Second First Third
- C) First Third Second
- D) Third First Second

3) Which of the following statements about Promises is correct?

A) A fulfilled promise can never become pending again

B) A rejected promise will execute the code in .then()

C) A promise can only be in two states: fulfilled or rejected

D) The only way to handle errors in promises is with `try...catch` blocks

Callback and Promises

MCQ



Answer 1 : C) `callback()`;

Answer 2: C) First Third Second

Answer 3: A) A fulfilled promise can never become pending again



THANK YOU

Prof. Pavan A C

Department of Computer Science and Engineering



WEB TECHNOLOGIES

XML vs. JSON

Prof. Pavan A C

Department of Computer Science and Engineering

Acknowledgement:

Teaching Assistants(Harini B and Chandana MS)

- XML stands for “extensible markup language”.
- XML was designed in 1996 and officially became a W3C standard in 1998.
- It was created to better represent data formats with a hierarchical structure.
- XML
 - Structures, stores, and transfers information and describes what the data is.
- HTML
 - Display data and focus on how the data looks.

- This data format fully supports hierarchical data structures and is very appropriate when receiving complex data as a response.
- It is also very human readable. Most browsers have built in XML readers that allow you to inspect XML files.
- Since XML was the first standard hierarchical data format, most APIs have built in functionality to automatically convert XML data streams into native data structures like objects.

- This data format is about three times as large as CSV. This is because each data element has an associated open and close parameter tag.

XML vs. JSON

JSON

- JSON stands for (Javascript Object Notation).
- It was invented in 2001 and became popularized by Yahoo and Google in 2005 and 2006.
- It was created as an alternative to XML. Like XML, however, it represents hierarchical data with the use of commas, curly braces and brackets.

XML Vs JSON HTML5

JSON PROS



- This data format supports hierarchical data while being smaller in size than XML.
- As its name implies, it was also created to more easily **parse data into native Javascript** objects, making it very useful for web applications.
- JSON is the best of both worlds with respect to CSV and XML. It's simple and compact like CSV, but supports hierarchical data like XML. Unlike XML, JSON formats are only about twice as large as CSV formats.

XML Vs JSON HTML5

JSON CONS

- This data format has a little bit less support than XML.
- Since JSON is relatively newer than XML, fewer APIs exist to automatically convert JSON to native data structures.
- However, this is rapidly changing because newer APIs and plugins are supporting both XML and JSON.

XML

```
<empinfo>
  <employees>
    <employee>
      <name>James Kirk</name>
      <age>40</age>
    </employee>
    <employee>
      <name>Jean-Luc Picard</name>
      <age>45</age>
    </employee>
    <employee>
      <name>Wesley Crusher</name>
      <age>27</age>
    </employee>
  </employees>
</empinfo>
```

JSON

```
{ "empinfo" :
  {
    "employees" : [
      {
        "name" : "James Kirk",
        "age" : 40,
      },
      {
        "name" : "Jean-Luc Picard",
        "age" : 45,
      },
      {
        "name" : "Wesley Crusher",
        "age" : 27,
      }
    ]
  }
}
```

XML vs. JSON

Differences

JSON	XML
Based on JavaScript language	Derived from SGML
Way of representing objects	Markup language to represent data items
No support for namespaces	Supports namespaces
Support arrays	Doesn't support arrays
Files are very easy to read as compared to XML	Documents are comparatively difficult to read and interpret
Doesn't use end tag	Has start and end tags
Less secure	More secured than JSON
Doesn't supports comments	Supports comments
Supports only UTF-8 encoding	Supports various encoding

XML Vs JSON HTML5

JSON

JSON String

```
'{"name":"John", "age":30, "car":null}'
```

XML Vs JSON HTML5

JSON

- JavaScript has a built in function for converting JSON strings into JavaScript objects:
`JSON.parse()`
- JavaScript also has a built in function for converting an object into a JSON string:
`JSON.stringify()`
- You can receive pure text from a server and use it as a JavaScript object.
- You can send a JavaScript object to a server in pure text format.
- You can work with data as JavaScript objects, with no complicated parsing and translations.

A text string is defined:

```
text = "<bookstore><book>" +  
"<title>Everyday Italian</title>" +  
"<author>Giada De Laurentiis</author>" +  
"<year>2005</year>" +  
"</book></bookstore>";
```

Parses a text string into an XML DOM object, and extracts the info from it with JavaScript:

An XML DOM parser is created:

```
parser = new DOMParser();
```

The parser creates a new XML DOM object using the text string:

```
xmlDoc = parser.parseFromString(text, "text/xml");
```

XML DOM object to XML string:

XMLSerializer interface provides `serializeToString()` method to construct an XML string

```
var oSerializer = new XMLSerializer();  
    var sXML = oSerializer.serializeToString(xmlDoc);  
    console.log(sXML);
```

1. Which of the following is the correct JavaScript function to convert a JSON string into a JavaScript object?

- A) JSON.stringify()
- B) parseJSON()
- C) JSON.parse()
- D) JSON.toObject()

2. In the following code, what will be the output in the console?

```
javascript
```

```
let person = { "name": "John", "age": 30 };
```

```
console.log(JSON.stringify(person));
```

- A) [object Object]
- B) {"name":"John","age":30}
- C) undefined
- D) name=John,age=30

XML Vs JSON HTML5

MCQ



3. Which statement about XML is true?

- A) XML does not support comments
- B) Every XML element must have both a start and end tag
- C) XML supports arrays natively
- D) XML cannot be read by browsers

4. Using JavaScript and the DOMParser, which line successfully parses an XML string `str` into an XML DOM object?

- A) `let obj = JSON.parse(str, 'application/xml');`
- B) `let obj = new XMLSerializer().parseFromString(str, 'text/xml');`
- C) `let obj = (new DOMParser()).parseFromString(str, 'text/xml');`
- D) `let obj = parseDOM(str, 'text/xml');`

XML Vs JSON HTML5

MCQ Answers

- Answer 1: C) `JSON.parse()`
- Answer 2: B) `{"name":"John","age":30}`
- Answer 3: B) Every XML element must have both a start and end tag
- Answer 4: C) `let obj = (new DOMParser()).parseFromString(str, 'text/xml');`



THANK YOU

Prof. Pavan A C

Department of Computer Science and Engineering



WEB TECHNOLOGIES

Single Page Applications

Prof. Pavan A C

Department of Computer Science and Engineering

Acknowledgement:

Teaching Assistants(Harini B and Chandana MS)

Definition: A **Single Page Application** loads a single HTML page, then dynamically updates it with JavaScript as users interact. No full reloads.

Key Point: It behaves more like a **native mobile app** than a traditional website.

Examples: Gmail, Netflix, Trello, Google Maps.

Why important: Faster, smoother UX -> today's standard for modern apps.

Analogy: It's like scrolling Instagram - you keep seeing new posts, but the app never reloads.

Initial Load: Browser gets HTML + JS + CSS in one shot.

Interactions: Future data requests go via **AJAX / Fetch / GraphQL APIs**, updating only parts of the DOM.

Under the Hood: Uses client-side routing (via `history.pushState` or frameworks like React Router).

Outcome: Pages feel seamless, transitions are instant.

Tech Note: React, Angular, Vue are the most common SPA frameworks today.

- **Speed:** Faster after first load -> less bandwidth used.
- **App-like UX:** Smooth transitions, no page flicker.
- **Separation of Concerns:** Backend provides APIs; frontend handles rendering.
- **Developer Benefits:** Easier to build modular UIs, better reuse of components.
- Nobody likes waiting... SPAs are like 2x speed on YouTube but for websites :)

SEO: Dynamic rendering makes it hard for crawlers (workarounds: SSR, pre-rendering).

Routing/History: Back/forward buttons don't work automatically -> need JS libraries.

Performance: Initial bundle can be heavy (100s of KBs/MBs).

Security: Client-side code = more exposed attack surface.

Definition: A **Multi-Page Application** loads a **new HTML page** from the server every time the user navigates or submits something.

Workflow:

1. User clicks -> request sent to server
2. Server processes -> sends back a new HTML page
3. Browser replaces the old page with the new one

Examples: Old portals (IRCTC, SBI net banking), news sites, blogs.

Strengths:

- SEO-friendly (pages are easy for search engines to crawl).
- Easier security handling (logic mostly on the server).
- Good for content-heavy sites.

Limitations:

- Slower UX (page reloads each time).
- More bandwidth consumption.
- Choppier navigation.

Think of an MPA as a waiter who brings you a new plate every time you order something, instead of just topping up your plate (which is what SPA does).

Extra Info

SPA vs MPA

Feature	SPA	Traditional MPA
Reloads	None after first load	Full reload on every click
Speed	Instant after load	Slower, multiple round trips
SEO	Needs extra handling	Works out of the box
UX	Seamless, app-like	Choppy, page-to-page
Dev Model	API + JS rendering	Server renders HTML each time

Real World: Flipkart web app = SPA-like; old government portals = MPAs.

SPAs shine in:

- Email apps (Gmail, Outlook)
- Project boards (Trello, Jira)
- Streaming (Netflix, Hotstar)
- Dashboards & analytics apps

Not ideal for:

- Blogs, news sites, or anything SEO-heavy.

Mnemonic: If it's interactive -> SPA; if it's mostly static info -> MPA.

- **Optimize Initial Load:** Use code splitting, lazy loading.
- **Handle SEO:** SSR (Next.js, Nuxt.js) or pre-rendering.
- **Routing:** Use libraries like React Router or Angular Router.
- **Caching:** Service Workers for offline/PWA experience.
- **Security:** Sanitize inputs, avoid exposing secrets in client code.
- **Tip:** Always test SPAs on 3G/4G networks -> real-world users won't always have WiFi.

- [Wikipedia – Single Page Application](#)
- [Bloomreach – What is a Single Page Application?](#)
- [Adobe Business Blog – Benefits of SPAs](#)
- [Netguru – SPA Pros & Cons](#)
- [Digiteum – SPA Features & Use Cases](#)
- [NetSolutions – SPA Insights & Architecture](#)
- [Medium – SPA vs MPA Comparison](#)
- [Wired – SPA Routing & History Issues](#)
- [StackOverflow – SPA Advantages & Disadvantages](#)

Q1. What is the main characteristic of a Single Page Application (SPA)?

- a) Each user action reloads the entire page
- b) Only one HTML page is loaded, content updates dynamically
- c) Every request renders a new HTML page on the server
- d) It works only with PHP backends

Answer: b) Only one HTML page is loaded, content updates dynamically

Q2. Which of the following is a *challenge* in building SPAs?

- a) Smooth user experience
- b) Back/forward navigation handling
- c) Faster interactions after first load
- d) Clear separation of frontend & backend

Answer: b) Back/forward navigation handling

Q3. Which technology or feature is most commonly used to fetch data in SPAs without reloading the page?

- a) HTML forms
- b) AJAX / Fetch API
- c) PHP sessions
- d) Cookies

Answer: b) AJAX / Fetch API



THANK YOU

Prof. Pavan A C

Department of Computer Science and Engineering



WEB TECHNOLOGIES

Async / Await

Prof. Pavan A C

Department of Computer Science and Engineering

Acknowledgement:

Teaching Assistants(Harini B and Chandana MS)

- **Async and Await in JavaScript** are used to simplify handling asynchronous operations using promises. By enabling asynchronous code to appear synchronous, they enhance code readability and make it easier to manage complex asynchronous flows.
- The **async** function allows us to write promise-based code as if it were synchronous. This ensures that the execution thread is not blocked.
- The keyword **async** before a function makes the function return a promise:

```
async function Function1() {  
    return "Returns a promise";  
}
```

Async / Await

Introduction

```
const getData = async () => {  
    let data = "Hello World";  
    return data;  
}
```

```
getData().then(data => console.log(data));
```

- If a function returns a value, the Promise is resolved with that value.
- If an exception is thrown, the Promise is rejected.

- The async function `getData` returns "Hello World" wrapped in a Promise.
- `.then()` is used to get the resolved result and print it.
- Output: Hello World

The await keyword is used to wait for a promise to resolve. It can only be used within an async block. Await makes the code wait until the promise returns a result

- It pauses the async function until the Promise resolves/rejects.
- Returns the resolved value or throws an error if the Promise is rejected.

```
async function getData() {  
    let response = await  
    fetch("https://api.example.com/data");  
    let data = await response.json();  
    return data;  
}
```

// Practical example

```
async function fetchUser(userId) {  
  try {  
    const response = await fetch(`https://jsonplaceholder.typicode.com/users/${userId}`);  
    const user = await response.json();  
    console.log(user);  
  } catch (error) {  
    console.error("Failed to fetch user:", error);  
  }  
}  
fetchUser(1);
```

```
async function getUserAndPosts(userId) {  
  try {  
    const userResponse = await fetch(`https://jsonplaceholder.typicode.com/users/${userId}`);  
    const user = await userResponse.json();  
    const postsResponse = await fetch(`https://jsonplaceholder.typicode.com/posts?userId=${userId}`);  
    const posts = await postsResponse.json();  
    console.log(user);  
    console.log(posts);  
  } catch (error) {  
    console.error('Error:', error);  
  }  
}  
getUserAndPosts(1);
```

Sequential API
calls

```
async function fetchMultiple() {  
  try {  
    const [user, post] = await Promise.all([  
      fetch('https://jsonplaceholder.typicode.com/users/1').then(res => res.json()),  
      fetch('https://jsonplaceholder.typicode.com/posts/1').then(res => res.json())  
    ]);  
    console.log('User:', user);  
    console.log('Post:', post);  
  } catch (err) {  
    console.error('Failed to fetch:', err);  
  }  
}  
fetchMultiple();
```

Promise.all for
parallel requests

```
function wait(ms) {  
    return new Promise(resolve => setTimeout(resolve, ms));  
}
```

```
async function demoDelay() {  
    console.log('Waiting...');  
    await wait(2000);  
    console.log('2 seconds later!');  
}  
demoDelay();
```

- `demoDelay` is an async function.
- It first logs `"Waiting..."` immediately.
- Then it `awaits` the `wait(2000)` call, pausing execution inside the function until the Promise resolves after 2000 milliseconds (2 seconds).
- After waiting 2 seconds, it proceeds to log `"2 seconds later!"`.

O/P

Waiting...

(2 second pause)

2 seconds later!

Error Handling with `try / catch` in Async Functions

- When using `async / await`, errors from awaited Promises can be caught just like synchronous code errors.
- Wrapping your awaited code inside a `try` block lets you handle errors gracefully in the accompanying `catch` block.
- This approach avoids scattered `.catch()` calls and makes your async error handling clearer and easier to manage.
- If any awaited Promise rejects, control jumps to `catch`, enabling centralized error processing.

Similar to the above example

Async / Await

Multiple Await

Multiple Await Statements

```
async function process() {  
  let a = await doA();  
  let b = await doB(a);  
  let c = await doC(b);  
  return c;  
}
```

```
async function getBoth() {  
  let [user, posts] = await Promise.all([  
    fetchUser(),  
    fetchPosts()  
  ]);  
  // both fetched in parallel  
}
```

Sequential
Parallel

Best Practices

- Always use try/catch for error handling.
- Avoid blocking the main thread—`await` is for I/O network, not CPU-heavy tasks.
- Use `async/await` for serial operations, `Promise.all` for parallel.
- Every `async` function returns a Promise.

Real-World Use Cases

- Fetching API data in web apps
- Database queries in Node.js servers
- Reading files asynchronously
- Chained and dependent HTTP requests

1. What will "async function example() { return 5; }" return when called?
 - A) 5
 - B) A Promise that resolves with 5
 - C) null
 - D) undefined
2. What happens if a Promise awaited with `await` rejects, and the code is NOT inside a try/catch?
 - A) The function stops without error
 - B) JavaScript ignores the error
 - C) The async function returns a rejected Promise with that error
 - D) The function returns undefined

3. Which of the following is a correct usage of await?

- A) At the top level in any JavaScript file
- B) Only inside an async function
- C) Inside any normal function
- D) Inside setTimeout

4. To execute three API calls in parallel and wait for all to finish, which is the best approach?

- A) Placing three await statements in sequence
- B) Using Promise.all([...]) around the three Promises with await
- C) Running three asynchronous functions in three scripts
- D) Using only callbacks

1. **Answer 1:** B) A Promise that resolves with 5
2. **Answer 2:** C) The async function returns a rejected Promise with that error
3. **Answer 3:** B) Only inside an `async` function
4. **Answer 4:** B) Using `Promise.all([...])` around the three Promises with `await`



THANK YOU

Prof. Pavan A C

Department of Computer Science and Engineering



WEB TECHNOLOGIES

AXIOS

Prof. Pavan A C

Department of Computer Science and Engineering

Acknowledgement:

Teaching Assistants(Harini B and Chandana MS)

- Axios is a popular open-source JavaScript library used to make HTTP requests from web browsers or Node.js environments.
- It simplifies the process of sending asynchronous HTTP requests to REST endpoints and handling responses.
- Features :
 - Promise-based API.
 - Works in both NodeJS and browsers.
 - Automatically transforms JSON data.
 - Supports request and response interceptors.
 - Allows easy handling of timeouts and cancellation of requests.
 - Supports making GET, POST, PUT, DELETE, and other HTTP requests.

- **Installing via npm**

- To install Axios in a Node.js project, run:

```
npm install axios
```

- After installing axios can be imported by including the below code.

```
import axios from 'axios';
```

- **Including in HTML**

- For use in a browser environment, include Axios via a CDN link in your HTML file:

```
<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"> </script>
```

Making a GET Request

```
const axios = require('axios');  
// Making a GET request  
axios.get('https://exampleget.com/data')  
  .then(response => {  
    console.log(response.data);  
  })  
  .catch(error => {  
    console.error('Error fetching data:', error);  
  });
```

Here if there is a correct response the content is logged or else error is logged.

Sending POST Requests

```
axios.post('https://examplepost.com/data', {  
  name: Name,  
  age: 20  
})  
  
.then(response => {  
  console.log(response.data);  
})  
  
.catch(error => {  
  console.error(error);  
});
```

Here if there is a correct response the content is logged or else error is logged.

Errors can be handled easily with the catch block. Any number of logging statements or actions can be taken

Apart from the basic GET and POST requests, Axios also supports a variety of advanced request options and features, making it a powerful and flexible HTTP client.

1. Common Request Options

- Setting Headers: Customize HTTP headers for your requests, such as Authorization tokens, content types, or custom headers.
- Query Parameters (**params**): Easily append query strings to your URLs by providing an object of key-value pairs, making your GET requests more dynamic.
- Timeouts: Define a maximum time to wait for a server response, so your app doesn't hang if a request takes too long.
- Response Types: Specify the expected format of the response data like JSON, text, blob, or stream, allowing appropriate handling.

// EXAMPLE

```
axios.get('/user', {  
  params: { id: 123 },  
  headers: { 'Authorization': 'Bearer your_token' },  
  timeout: 5000,  
  responseType: 'json'  
})  
  .then(response => {  
    console.log(response.data);  
  })  
  .catch(error => {  
    console.error('Request failed:', error);  
  });
```

2. Handling Responses

Axios makes it straightforward to access parts of the HTTP response:

- `response.data` contains the response payload (body).
- `response.status` is the HTTP status code (e.g., 200 OK, 404 Not Found).
- `response.headers` holds the response headers.

3. Global Configuration

You can configure Axios to have default settings that apply to all requests made via an instance like timeout or base URL. This reduces repetition

Practical Best Practices and Real-world Use-cases for Axios

- Always handle errors: Use `.catch()` or `try-catch` with `async/await` to gracefully handle network or server errors and show user-friendly messages.
- Validate response data: Verify the structure and type of data received before using it
- Use `async/await` syntax: Use `async` functions and `await` instead of chaining `.then()`.

Real-World Use-cases:

- RESTful API calls from popular frontend frameworks such as React, Vue, or Angular to fetch and send data.
- Server-side data fetching in Node.js environments (e.g., for server-side rendering or backend services) using Axios for consistent HTTP request handling.

1. Which of the following is NOT a common request option supported by Axios?
 - A) Setting HTTP headers
 - B) Using query parameters (**params**)
 - C) Defining request timeouts
 - D) Automating UI refresh
2. Which property of the Axios response object contains the main data sent from the server?
 - A) **response.status**
 - B) **response.config**
 - C) **response.data**
 - D) **response.headers**

3. How can you globally set a base URL for all Axios requests?

- A) By calling `axios.setBaseURL()` method
- B) Using `axios.defaults.baseURL` property
- C) Assigning `axios.base` property
- D) Globally configuring `axios.config`

4. Which of the following statements about Axios is correct?

- A) Axios can only be used in Node.js environments, not in browsers
- B) Axios automatically transforms JSON data in HTTP requests and responses
- C) Axios requires callbacks instead of Promises for asynchronous code
- D) Axios cannot set custom HTTP headers in requests

Answer 1: D) Automating UI refresh

Answer 2: C) `response.data`

Answer 3: B) Using `axios.defaults.baseURL` property

Answer 4 : B) Axios automatically transforms JSON data in HTTP requests and responses



THANK YOU

Prof. Pavan A C

Department of Computer Science and Engineering

React Setup & Installation (npm)

What is npm?

- **NPM (Node Package Manager)** is the default package manager for **Node.js**.
- Used to:
 - o Manage **project dependencies** (third-party libraries).
 - o Run **scripts** (e.g., start server, build app).
 - o Share and reuse **packages/modules**.
- Installing Node.js also installs npm automatically.

A **package** = a reusable module of code (from a small utility to a full-featured library).
Installed packages are stored in the **node_modules** folder.

Installing Node.js & npm

1. Download & install Node.js from:
[Node.js Installation Guide](#)
2. During installation → Add Node.js to **PATH**.
3. Verify installation:
 1. `node -v`
 2. `npm -v`

Setting up a React Project

```
npm create vite@latest my-react-app -- --template react
cd my-react-app
npm install          (Install all dependencies)
npm run dev
```



WEB TECHNOLOGIES

ReactJS - MERN Introduction

Prof. Pavan A C

Department of Computer Science and Engineering

Acknowledgement:

Teaching Assistants(Harini B and Chandana M S)

Web Development Stack

The Big Picture



Definition: A collection of technologies (frontend, backend, database) working together to build modern web apps.

Fun Analogy: Think of a web dev stack as a **restaurant setup**:

- **Frontend (UI)** = the **menu & dining area** (what customers see).
- **Backend (Server)** = the **kitchen** (where the magic happens).
- **Database** = the **fridge/pantry** (where ingredients/data are stored).

Just like a restaurant needs all 3 to serve food, a web app needs all 3 to serve functionality.

Web Development Stack

Full Web Development Stack



A **full stack** = covering **frontend + backend + database**.

Popular stacks:

- MERN Stack
- MEAN Stack
- MEVN Stack
- Serverless Stack
- LAMP Stack
- PERN Stack
- Ruby on Rails Stack

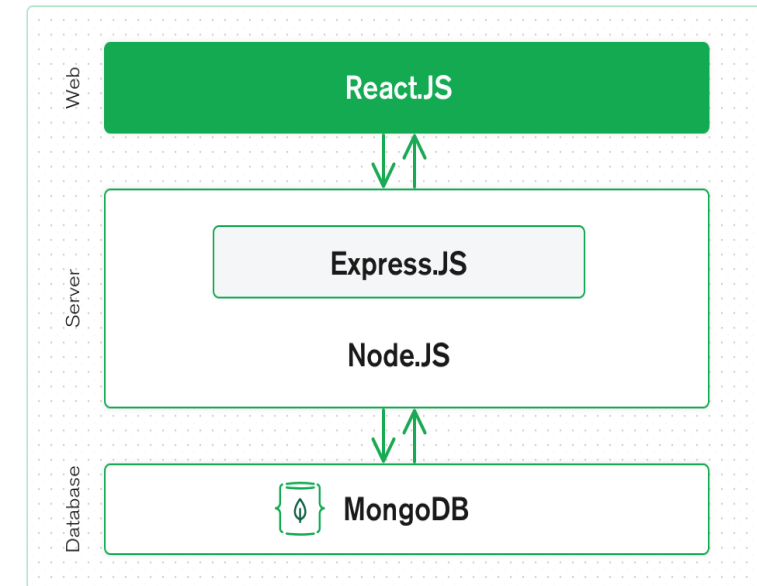
Analogy continuation: A **full restaurant team** — waiters (frontend), chefs (backend), and pantry managers (database). Without one, the restaurant breaks down.

Web Development Stack

MERN Stack Introduction

MERN = MongoDB + Express.js + React.js + Node.js.

- MongoDB - document database
- Express(.js) - Node.js web framework
- React(.js) - a client-side JavaScript framework
- Node(.js) - the premier JavaScript



100% JavaScript across frontend, backend, and database communication.

Analogy: Imagine running a restaurant where **everyone speaks the same language (say, English)** — smooth communication, fewer errors. That's MERN for devs.

ReactJS (The Frontend Library)

- Top tier of MERN stack.
- A **component-based JS library** for building UIs.
- Handles dynamic, data-driven views efficiently with Virtual DOM.
- **Analogy:** React is the **waiter** — presenting dishes (data) nicely to customers (users), updating the table whenever the order changes.

NodeJS (The Runtime Environment)

- Middle tier of MERN stack.
- A **JavaScript runtime** that runs code on the server side.
- Powers Express, handles requests, connects to DB, runs logic.
- **Analogy:** Node.js is the **kitchen** itself — the environment where cooking (execution) happens. Without the kitchen, the chef can't cook.

ExpressJS (The Server Framework)

- Middle tier of MERN stack.
- A lightweight **NodeJS framework** for handling routes & HTTP requests.
- Simplifies server logic with middleware & routing.
- **Analogy:** Express is the **chef assistant** — it makes sure the kitchen (Node) follows recipes correctly and serves dishes quickly.

MongoDB (The Database)

- Bottom tier of MERN stack.
- A **NoSQL database** storing data as documents (JSON-like).
- Flexible, schema-less, great for dynamic apps.
- **Analogy:** MongoDB is the **pantry/fridge** of the restaurant, it doesn't enforce rigid "shelves," you can store ingredients in whatever containers you like.

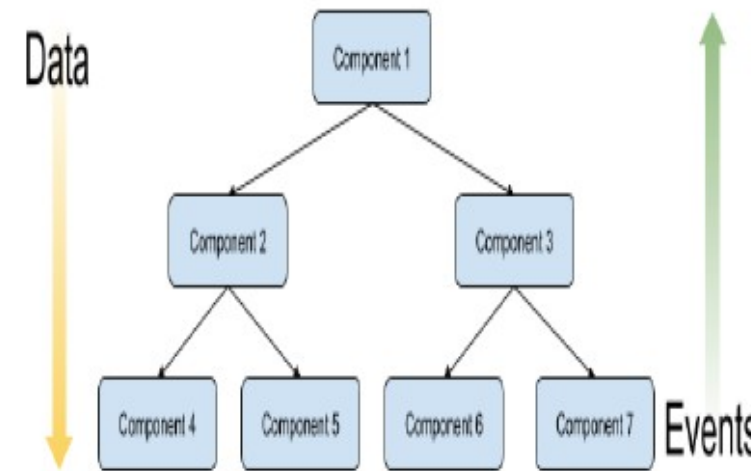
- **What:** Open-source JavaScript library created by Facebook in 2013.
- **Why:** Enables building **single-page apps (spa)** with fast, dynamic updates, without the need of reloading.
- **Core Idea:** UI = components + state + props.
- **Fun Angle:** React is like **LEGO blocks** — build small reusable pieces (components), combine them into big structures (UIs).

- **Declarative** → You tell React *what* you want, it figures out *how*.
- **Simple** → Clear, focused, readable.
- **Component-based** → Everything reusable, modular.
- **Supports server side** → Rendering can also happen on the server.
- **Mobile support** → Via React Native.
- **Extensive** → Huge ecosystem & libraries.
- **Fast** → Optimized updates via Virtual DOM.
- **Easy to learn** → JavaScript + JSX = beginner-friendly.
- **Single-way data flow** → Props down, actions up.
- **Virtual DOM** → Efficient rendering by comparing virtual vs real DOM.

Definition: React enforces a **unidirectional data flow**, meaning data always flows from parent → child components.

Mechanism:

- **Props:** Passed down from parent to child (read-only).
- **Callbacks / State Updaters:** Child can communicate changes *up* by invoking functions provided by the parent.
- Actions flow up and properties flow down



Why this matters:

- Predictable data handling — you always know the source of truth.
- Easier debugging — data changes can be traced step by step.
- Better maintainability — prevents unexpected side-effects from two-way binding.

Analogy: Like a **waterfall** — water (data) flows down from the top (parent), but if something at the bottom (child) wants to influence the top, it sends a **signal** (callback) back up.

Definition: The **Virtual DOM (VDOM)** is an in-memory lightweight copy of the actual DOM. React updates this virtual representation first before syncing it with the real DOM.

How it works (Reconciliation Process):

1. React creates a **virtual DOM tree** when you write JSX.
2. On state/prop change, React generates a **new VDOM tree**.
3. React performs a **diffing algorithm** to compare old vs new VDOM.
4. Only the **minimal set of changes** are applied to the real DOM.

Why this matters:

- The real DOM is **expensive** to manipulate — each change can trigger re-layouts, re-paints, and re-flows.
- The VDOM reduces these costs by batching & optimizing updates.
- Results in **better performance** and a smoother user experience.

Analogy:

Imagine redesigning your **room**:

- Instead of tearing everything down, you **sketch changes on paper first** (Virtual DOM).
- Once you know what's different (diffing), you only **move the table** or **change the poster**, instead of repainting the entire room (real DOM)

Way 1: Using Node Package Manager (npm)

```
npx create-react-app my-app
```

```
cd my-app
```

```
npm start
```

Way 2: Directly import React in HTML

```
<script src="https://unpkg.com/react@18/umd/react.development.js"></script>
```

```
<script      src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
```

Q1. In React, data flows in which direction by default?

- a) Two-way binding
- b) Upwards only
- c) Single-way (Parent → Child)
- d) Random

Answer: c) Single-way (Parent → Child)

Q2. In the MERN stack, which component is responsible for serving as the runtime environment?

- a) MongoDB
- b) Express.js
- c) React.js
- d) Node.js

Answer: d) Node.js

Q3. Which feature of React ensures faster rendering by minimizing real DOM updates?

- a) Babel
- b) JSX
- c) Virtual DOM
- d) Components

Answer: c) Virtual DOM

- [React Official Docs – react.dev](https://react.dev)
- [MDN Web Docs – Getting started with React](https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Modern_frameworks/React)
- [W3Schools React Intro](https://www.w3schools.com/react/)
- [MongoDB MERN Overview](#)
- [GeeksforGeeks – MERN Stack](https://www.geeksforgeeks.org/mern-stack/)
- [Contentful Blog – MERN Stack](#)
- [Oracle – MERN vs Other Stacks](#)



THANK YOU

Prof. Pavan A C

Department of Computer Science and Engineering



WEB TECHNOLOGIES

JSX-Rendering Of Elements

Prof. Pavan A C

Department of Computer Science and Engineering

Acknowledgement:

Teaching Assistants(Harini B and Chandana MS)

JSX - Rendering Of Elements

What is JSX



JSX stands for JavaScript XML, and it is a special syntax used in React to simplify building user interfaces.

- JSX allows you to write HTML-like code directly inside JavaScript, enabling you to create UI components more efficiently.
- Although JSX looks like regular HTML, it's actually a syntax extension for JavaScript.

Although there are other ways to write components, most React developers prefer the conciseness of JSX, and most codebases use it.

Example :

```
const element = <h1>Hello, JSX!</h1>;
```

JSX - Rendering Of Elements

How JSX Works



- JSX code is compiled/transformed into `React.createElement()` calls by tools like Babel.
- Compiled JSX creates plain JavaScript objects called React elements.

```
const element = <h1>Hello, world!</h1>;  
// tools like Babel Compiles to:  
const element = React.createElement('h1', null, 'Hello,  
world!');
```

The JSX code `<h1>{message}</h1>` will be transformed into JavaScript by Babel. Then react will then create a virtual DOM element for the `<h1>` tag with the text inside. and this virtual DOM is then used to update the actual browser DOM, displaying "Hello, world!" on the screen.

JSX - Rendering Of Elements

Converting HTML to JSX



- React components use JSX to define UI markup inside JavaScript.
- HTML looks similar but JSX has stricter syntax rules.
- Direct copy-pasting HTML into JSX often causes errors

Rules for converting HTML to JSX

- **Rule 1 – Single Root Element**
- **Rule 2 – Close All Tags**
- **Rule 3 – Use camelCase for Attributes**

JSX compiles to JavaScript objects; attribute names become object keys.

JavaScript has naming restrictions, hence camelCase.

Single root element needed for consistent element representation in React.

JSX - Rendering Of Elements

Converting HTML to JSX

Single Root Element

BEFORE

```
<h2>Tasks</h2>
<ul>
  <li>Debugging code</li>
  <li>Writing documentation</li>
  <li>Presenting talks</li>
</ul>
```

AFTER

```
return (
  <>
    <h2>Tasks</h2>
    <ul>
      <li>Debugging code</li>
      <li>Writing
documentation</li>
      <li>Presenting talks</li>
    </ul>
  </>
)
```

JSX must return a single parent element.

Wrap siblings inside a <div>, <section>, or React Fragment <>...</>.

JSX - Rendering Of Elements

Converting HTML to JSX

Attribute Names

BEFORE

```

<ul>
  <li>Presenting talks</li>
</ul>
```

AFTER

```
return (
  <>
    
    <ul>
      <li>Presenting talks</li>
    </ul>
  </>
);
```

JSX attributes are **camelCase**, not HTML attribute names.

- Use className not class
- Use htmlFor not for
- Use tabIndex instead of tabindex

JSX - Rendering Of Elements

Embedding Expressions

In the example below, we declare a variable called name and then use it inside JSX by wrapping it in curly braces:

```
const name = 'Josh Perez';  
const element = <h1>Hello, {name}</h1>;
```

You can put any valid JavaScript expression inside the curly braces in JSX. After compilation, JSX expressions become regular JavaScript function calls and evaluate to JavaScript objects.

```
function getGreeting(user) {  
  if (user) {  
    return <h1>Hello, {formatName(user)} !</h1>;  
  }  
  return <h1>Hello, Stranger.</h1>;  
}
```

This means that you can use JSX inside of if statements and for loops, assign it to variables, accept it as arguments, and return it from functions:

JSX - Rendering Of Elements

Embedding Expressions

```
const user = { name: 'John' };  
const element = <h1>Welcome, {user.name.toUpperCase()}!</h1>;
```

```
const isLoggedIn = true;  
const message = <h1>{isLoggedIn ? 'Logout' : 'Login'}</h1>;
```

```
const numbers = [1, 2, 3];  
const listItems = numbers.map(num => <li  
key={num.toString()}>{num}</li>);  
const element = <ul>{listItems}</ul>;
```

Simple Rendering Example

Render an element to DOM using React 18+ API:

```
const root =  
ReactDOM.createRoot(document.getElementById('root'));
```

JSX Spread Attributes

Spread operator to pass props dynamically:

```
const props = { name: 'Sara', age: 30 };  
const element = <User {...props} />;
```

Use function variables:

```
function renderContent(isLoading) {  
  if (isLoading) return <LoadingSpinner />;  
  return <Content />;  
}
```

Handling Events in JSX

React uses camelCase event naming:

```
<button onClick={handleClick}>Click</button>
```

JSX - Rendering Of Elements

JSX Rendering - Example

```
const element = (  
  <div>  
    <h1>Hello, JSX!</h1>  
    <p>2 + 2 = {2 + 2}</p>  
  </div>  
);
```

```
ReactDOM.createRoot(document.getElementById("root")).render(element);
```

JSX - Rendering Of Elements

JSX Rendering - Example

```
const element = (  
  <div>  
    <h1>My Favorite Fruits</h1>  
    <ul>  
      <li>Apple</li>  
      <li>Mango</li>  
      <li>Banana</li>  
    </ul>  
  </div>  
);
```

```
ReactDOM.createRoot(document.getElementById("root")).render(element);
```

1. What is the correct way to set a CSS class in JSX?

- A) `<div class="container">`
- B) `<div className="container">`
- C) `<div classname="container">`
- D) `<div class-name="container">`

Answer: B

2. How would you embed the JavaScript expression `2 + 2` inside a JSX element?

- A) `<p>2 + 2</p>`
- B) `<p>{2 + 2}</p>`
- C) `<p>{{2 + 2}}</p>`
- D) `<p>{(2 + 2)}</p>`

Answer: B (also D is valid)

Which of the following is a valid self-closing JSX tag?

- A) `<img>`
- B) `<img />`
- C) `<img></img>`
- D) Both B and C

Answer: D

How do you embed a JavaScript expression in JSX?

- A) `{ }`
- B) `< >`
- C) `( )`

Answer: A

What will be the output of the following JSX code?

```
const show = false;
```

```
const element = <div>{show && <p>Hello, World!</p>}</div>;
```

- A) `<div><p>Hello, World!</p></div>`
- B) `<div></div>`
- C) `<div>false</div>`
- D) Syntax error

Answer: B) `<div></div>`

Since show is false, the logical AND (&&) expression will short-circuit and not render the `<p>` element, resulting in an empty `<div>`.



THANK YOU

Prof. Pavan A C

Department of Computer Science and Engineering



WEB TECHNOLOGIES

React Components and JSX

Prof. Pavan A C

Department of Computer Science and Engineering

Acknowledgement:

Teaching Assistants(Harini B and Chandana MS)

In React, components are reusable, independent code blocks (A function or a class) that define the structure and behavior of the UI. They accept inputs (props or properties) and return elements that describe what should appear on the screen.

Key Concepts of React Components:

- Each component handles its own logic and UI rendering.
- Components can be reused throughout the app for consistency.
- Components accept inputs via props and manage dynamic data using state.
- Only the changed component re-renders, not the entire page.
- Stateless by default (but can use hooks for state)
- Modern standard for React development

Need for Components

- There may be elements which are similar
- A need to make a change in one will reflect changes in multiple places
- Similar to functions, if we could write code related to the element in one place, changes can be minimized
- Solution - Reusable piece of JavaScript code that output (via JSX) HTML elements

- **Stateless Functional Component**

- Includes simple JavaScript functions and immutable properties, i.e., the value for properties cannot be changed.
- Use hooks to achieve functionality for making changes in properties using JS.
- Used mainly for UI.

- **Stateful Class Component**

- Classes which extend the Component class from React library.
- The class component must include the render method which returns HTML.

React Component and JSX (Functional Comps)

Functional Components

- The simplest way to define a component is to write a JavaScript function:
- A functional component is simply a JavaScript function that returns JSX (JavaScript XML, the syntax that looks like HTML inside JavaScript).
- They are the building blocks of modern React apps.

```
function Welcome() {  
  return <h1>Hello, World!</h1>  
}
```

Function Declaration or Arrow Functions
can be used.

JSX returned must be single root element.

```
// Arrow function syntax  
const Welcome = () => {  
  return <h1>Hello, World!  
    </h1>;  
}
```

React Component and JSX (Functional Comps)

Functional Components



Properties are ways in which React components can be customized.

- Props are arguments passed into React components and are passed via HTML attributes.
- Properties are immutable and are same as what attributes are to HTML elements.
- Their most basic use is in the form of attributes, in JSX.
- Are actually passed to a constructor and can be accessed by `this.props`
- Design decision is to use the Attributes specified in hyphen-case in HTML as camelCase in JSX.
- Make the function of your component read the props from the props parameter
- When rendering the component, add the prop to the component using the attribute

React Component and JSX (Functional Comps)

Functional Components

- Props (short for properties) are how you pass data into a component.
- props is an object that contains all the values passed to the component.

```
function Greeting(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}  
  
// Usage  
<Greeting name="Alice" />
```

This function is a valid React component because it accepts a single “props” (which stands for properties) object argument with data and returns a React element. We call such components “function components” because they are literally JavaScript functions.

React Component and JSX (Functional Comps)

Functional Components

Destructuring Props

Instead of using `props.attributename`, you can destructure the props object directly in the function's argument to make the code cleaner.

```
function UserCard({ name, email, age })  
  return (  
    <div>  
      <h2>{name}</h2>  
      <p>Email: {email}</p>  
      <p>Age: {age}</p>  
    </div>  
  );  
}
```

The UserCard component receives the name, email, age prop using destructuring in the function argument, making it easier to access the name value.

Instead of writing `props.name`, we now directly use `{ name }` within the component.

- Functional components are simple, reusable JavaScript functions that return JSX and accept props to customize their output.
- They are stateless by default but form the modern foundation of React apps, enabling modular UI and code reuse.
- In larger, interactive apps, components need to manage dynamic data, handle user actions, and perform side effects (like fetching data).

Introducing React Hooks

- React Hooks empower functional components to do more—use local state, respond to lifecycle events, and share logic easily.
- Hooks such as `useState` (for state) and `useEffect` (for side effects) unlock features that were previously only possible in class components.
- **Upcoming unit** will dive deep into hooks, and how they make functional components even more powerful for modern web development.

React Component and JSX (Functional Comps)

Functional Components - Example



```
function Student() {  
  const name: string = "Pavan";  
  const course: string = "Web Technologies";  
  const semester: number = 3;  
  
  return (  
    <div>  
      <h2>Student Information</h2>  
      <p>Name: {name}</p>  
      <p>Course: {course}</p>  
      <p>Semester: {semester}</p>  
    </div>  
  );  
}
```

```
// Render Student component  
ReactDOM.createRoot(document.getElementById  
("root1")).render(  
  
  <Student />  
  
  );
```

React Component and JSX (Functional Comps)

Functional Components - Example



```
function Welcome(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}
```

// Rendering the component

```
ReactDOM.createRoot(document.getElementById("root")).render(  
  <Welcome name="Pavan" />  
);
```

React Component and JSX (Functional Comps)

Functional Components - Example



```
function App() {  
  function handleClick() {  
    alert("Button was clicked!");  
  }  
  
  return (  
    <div>  
      <h1>Hello, JSX!</h1>  
      <p>2 + 2 = {2 + 2}</p>  
      <button onClick={handleClick}>Click Me</button>  
    </div>  
  );  
}
```

```
ReactDOM.createRoot(document.getEle  
mentById("root")).render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
);
```



THANK YOU

Prof. Pavan A C

Department of Computer Science and Engineering



WEB TECHNOLOGIES

React Component Styling

Prof. Pavan A C

Department of Computer Science and Engineering

Acknowledgement:

Teaching Assistants(Harini B and Chandana MS)

React Component Styling

Overview & Why It Matters



Definition: Styling in React refers to the various techniques used to apply visual design and presentation rules to individual components.

Importance: Effective styling improves both **user experience** and **code maintainability**, ensuring applications are consistent, accessible, and scalable.

Context: Unlike traditional web development, React encourages **component-scoped styling**, where each component manages its own appearance, reducing conflicts and improving modularity.

Why learn this: Choosing the right styling strategy can optimize performance, accelerate development, and enhance collaboration in larger projects.

Traditional Styling with CSS Files

External CSS (or CSS Modules)



- **How it works:** Create `.css` files and import them. Example with CSS Modules:

```
import styles from './Button.module.css';  
<button className={styles.primary}>Click Me</button>
```

- **Why use it:** Separation of concerns and scoped styles - no global naming mess.
- **Tip:** Keeps responsiveness manageable and clean by component splitting.
- **Analogy:** Like school uniforms – neat, consistent, no one fights over colors.

- **How** **it** **works:**

```
<div style={{ color: 'red', fontSize: '20px' }}>Hello Style!  
</div>
```
- **Pros:** Good for dynamic, one-off styles.
- **Cons:** Harder to maintain; lacks pseudo-class support (like `:hover`).
- **Analogy:** Like asking for 'extra masala' at a street food stall - quick, works for one plate, but not scalable for the whole restaurant.

CSS-in-JS (Styled Components & Beyond)

When CSS Meets JS—Introductions to CSS-in-JS



- **Concept:** Styles are defined in JS, using libraries like styled-components, Emotion, or JSS.

- **Example** with styled-components:

```
const Button = styled.button`  
  background: ${props => props.primary ? 'blue' : 'gray'};  
  color: white;  
`;
```

- **Why it's cool:** Encapsulation, theming, and no style collisions.
- **Analogy:** Like Swiggy Instamart - everything (groceries + spices) delivered together in one bag. Here, logic + styles live in one file.

- **TailwindCSS:** Write utility classes that compose styles - fast and modifiable.
- **Component Libraries:** Use ready-made components (Material UI, Chakra, Radix, Ant Design) with styling baked in and themeable.
- **Analogy:**
 - **(Utility-first):** Like using Zomato filters - add 'veg only', 'under ₹300', '4+ rating' - you stack utilities until you get exactly what you want.
 - **UI Libraries:** Like ordering from Domino's - ready-made, consistent taste, you just customize toppings (themes).

- **Hooks + CSS-in-JS + Feature Structure:** Keep your styling logic near code logic; modular and feature-aligned.
- **Use CSS Modules or CSS-in-JS** for isolation and maintainability.
- **Component-Driven Styles:** Each component owns its styles - clear and reusable.
- **Analogy:** Like splitting Netflix accounts - everyone has their own profile (scoped styles), but the subscription (project) stays organized.

React 19 & <style> Handling

React's Built-in Style Optimization



Behavior: React can optimize <style> tags:

- Moves them to <head>, de-duplicates duplicates.
- Honors href and precedence props to avoid conflicts.

-

Why care: Better performance and clearer override rules - no more random CSS chaos mid-app.

React Component Styling

Example - 1



```
function InlineStyled() {  
  return (  
    <div>  
      <h1 style={{ color: "blue", fontSize: "24px" }}>  
        Hello with Inline Style  
      </h1>  
      <p style={{ backgroundColor: "lightyellow",  
padding: "10px" }}>  
        This paragraph is styled using inline CSS in JSX.  
      </p>  
    </div>  
  );  
}
```

```
ReactDOM.createRoot(document.  
getElementById("root")).render(  
  <InlineStyled />  
);
```

React Component Styling

Example - 2



```
import React from "react";
import ReactDOM from "react-dom/client";
import "./index.css"; // Import CSS file

function InternalStyled() {
  return (
    <div>
      <h1 className="heading">Hello with Internal
CSS</h1>
      <p className="paragraph">
        This paragraph is styled using an external CSS file.
      </p>
    </div>
  );
}
```

```
ReactDOM.createRoot(document.getEle
mentById("root")).render(
  <InternalStyled />
);
```

Note

This is just main.jsx
// src/main.jsx
CSS file is also required

React Component Styling

Example - 2 (Continued)

```
//src/index.css
```

```
.heading {  
  color: green;  
  font-size: 26px;  
  text-align: center;  
}
```

```
.paragraph {  
  background-color: lightgray;  
  padding: 12px;  
  border-radius: 5px;  
}
```

Picking the Right Strategy

Which Styling Option is *You*?



Scenario	Best Option	Why It's Lit
Simple UI, small project	CSS Modules or traditional CSS	Fast to set up, minimal overhead
Thematic, large app	CSS-in-JS (styled-components / Emotion)	Scoped, dynamic, easier theme changes
Utility-first fan	Tailwind + component libraries	Speedy, consistent, dev-friendly
Built for enterprise	UI libraries like MUI, Chakra, Radix	Accessibility, aesthetics, ready-made polish

Q1. Which React styling method uses JavaScript template literals to define styles?

- a) Inline CSS
- b) Styled Components
- c) CSS Modules
- d) Tailwind CSS

Answer: b) Styled Components

Q2. Tailwind CSS follows which styling philosophy?

- a) Component-driven
- b) Global styles
- c) Utility-first
- d) Inline

Answer: c) Utility-first

Q3. One key advantage of CSS Modules is:

- a) Styles are global and reusable everywhere
- b) Styles are scoped locally to a component
- c) Styles are written only in JavaScript
- d) Supports only class-based components

Answer: b) Styles are scoped locally to a component

References

- [React Official Docs – <style> component](#)
- [W3Schools – React CSS Styling](#)
- [Styled Components Official Docs](#)
- [Wikipedia – CSS-in-JS](#)
- [FreeCodeCamp – How to Style React Components](#)
- [Prismic – React Component Libraries](#)
- [Medium – React Best Practices 2025](#)
- [Dev.to – React Components 2025 Guide](#)



THANK YOU

Prof. Pavan A C

Department of Computer Science and Engineering